



Университет Бриггс-Янг
Архив с типендиатов BYU

Публикации факультета

2011-7

Обзор и руководство по разработке в крупных компаниях.

Организация

Джордан Б. Барлоу

Университет Индианы

Джастин Скотт Гибони

Университет Аризона

Марк Джеффри Кит

Запад Техас являющийся университет

Дэвид У. Уилсон

Вашингтон Состояние Университет

Райан М. Шотцлер

Следите за мной и другими работами на <https://scholarsarchive.byu.edu/>



Часть Business Commons

Дополнительных авторов смотрите на следующей странице.

Первоначальная ссылка на

публикацию Барлоу, Дж. Б., Гибони, Дж. С., Кит, М. Дж., Уилсон, Д. В., Шотцлер, Р., Лоури, П. Б. и Вэнс, А. (2011). Обзор и руководство по гибкой разработке в крупных организациях. Сообщения Ассоциации информационных систем, 29, стр. 25–44.

Цитата из архива ученых BYU

Барлоу, Джордан Б.; Гибони, Джастин Скотт; Кит, Марк Джеффри; Уилсон, Дэвид У.; Шотцлер, Райан М.; Лоури, Пол Бенджамин; и Вэнс, Энтони, «Обзор и руководство по гибкой разработке в крупных организациях» (2011). 5667. <https://scholarsarchive.byu.edu/facpub/5667>

Факультет Публикации

Эта рецензируемая статья предоставляется вам в бесплатный открытый доступ архивом BYU ScholarsArchive. Она была одобрена для публикации в разделе «Публикации преподавателей» уполномоченным администратором архива BYU ScholarsArchive. Для получения дополнительной информации обращайтесь по адресу ellen.amatangelo@byu.edu.

Авторы

Джордан Б. Барлоу, Джастин Скотт Гибони, Марк Джеффри Кит, Дэвид У. Уилсон, Райан М. Шуллер,
Пол Бенджамин Лоури и Энтони Вэнс

Communications of the Association for Information Systems

CAIS

Обзор и руководство по гибкой разработке в крупных организациях

Джордан Б. Барлоу

Кафедра технологий операций и принятия решений, Школа бизнеса Келли, Университет Индианы
jordy.barlow@gmail.com

Джастин Скотт Гибони

Кафедра систем управления информацией, Колледж менеджмента Эллера, Университет Аризоны

Марк Джеффри Кит

Кафедра компьютерной информации и управления решениями, Западно-Техасский университет A&M

Дэвид У. Уилсон

Факультет предпринимательства и информационных систем, Университет штата Вашингтон

Райан М. Шуцлер

Кафедра систем управления информацией, Колледж менеджмента Эллера, Университет Аризоны

Пол Бенджамин Лоури

Кафедра информационных систем Городского университета Гонконга

Энтони Вэнс

Кафедра информационных систем, Школа менеджмента Марриотта, Университет имени Бригама Янга

Abstract:

Эффективность гибких методологий разработки программного обеспечения постоянно обсуждается. Некоторые организации внедряют гибкие методы для повышения конкурентоспособности, оптимизации процессов и снижения затрат. Другие организации скептически относятся к пользе гибкой разработки. Крупные организации сталкиваются с дополнительной проблемой интеграции гибких методов с существующими стандартами и бизнес-процессами. Чтобы изучить влияние гибких методов разработки на крупные организации, мы анализируем и обобщаем научную литературу и теоретические данные по гибкой разработке программного обеспечения. Кроме того, мы структурируем нашу теорию и наблюдения в структуру рекомендаций для крупных организаций, расматривающих гибкие методологии. Основываясь на этой структуре, мы представляем рекомендации, предлагающие способы успешного внедрения гибких методов крупными организациями с учетом имеющихся процессов. Наш анализ литературы и теории открывает новые горизонты для исследователей гибкой разработки программного обеспечения и помогает практикам определить, как внедрить гибкую разработку в своих организациях.

Ключевые слова: гибкость, гибкая разработка, разработка программного обеспечения, жизненный цикл, крупные организации, каскадный метод, экстремальное программирование, Scrum, неформальное общение, взаимозависимости, координация

Том 29, статья 2, с. 25-44, июль 2011 г.

I. ВВЕДЕНИЕ

Гибкие методы разработки программного обеспечения (agile) вызывают постоянные споры. Гибкая разработка программного обеспечения (agile) — это методология, которая включает итеративную разработку, частые консультации с заказчиком, небольшие и частые релизы и тщательно протестированный код [Сао et al., 2009]. Некоторые организации внедряют гибкие методы для повышения конкурентоспособности, оптимизации процессов и снижения затрат. Другие организации скептически относятся к преимуществам гибкой разработки. Крупные организации сталкиваются с дополнительной проблемой интеграции гибких методов с существующими стандартами и бизнес-процессами.

Разработчики программного обеспечения создали гибкую разработку, главным образом, для устранения недостатков методов, основанных на планировании, таких как влиятельный каскадный метод [Royce, 1970]. Основной недостаток методов, основанных на планировании, — недостаточная способность реагировать на изменения. В связи с последовательной природой разработки, основанной на плане, разработчики, использующие эту методологию, усугубляют требования и планируют проекты на ранних этапах процесса, что приводит к снижению гибкости на последующих этапах разработки. Однако требования к проекту часто существенно меняются между началом и завершением. Гибкая разработка позволяет организациям адаптироваться к этим динамичным условиям, способствуя более гибкой разработке [Cockburn and Highsmith, 2001].

Однако критикуются и преимущества гибкой разработки, перевешивая издержки [Ambler, 2008; Rising and Janoff, 2000; Selic, 2009]. Большинство из них указывает на недостаточное внимание к планированию и внедрению, а также на чрезмерную осведомленность на кодировании [Ambler, 2008]; на отсутствие необходимой документации, что часто является следствием недостаточного формального взаимодействия [Selic, 2009]; и на сбои в реализации крупных и сложных проектов [Rising and Janoff, 2000].

Чтобы рассмотреть этот вопрос с точки зрения крупных организаций, мы проанализировали и обобщили литературу по гибкой разработке программного обеспечения. Мы представляем краткое описание гибкой разработки программного обеспечения, её общих сильных и слабых сторон, а также организационных изменений, необходимых для внедрения гибких методов. Во многих научных статьях обсуждаются преимущества и недостатки гибкой разработки в небольших или моделируемых средах. Однако влияние гибкой разработки на крупные организации изучено лишь в редких случаях. Кроме того, лишь немногие исследователи используют теоретический подход для изучения причин успехов или неудач гибких методологий в крупных организациях. В данной статье мы представляем теоретическую структуру и рекомендации, которые указывают на возможные пути внедрения гибких методов крупными организациями с учетом вышесказанного.

Наш обзор литературы, а также теоретическая структура и сопутствующие рекомендации дают представление исследователям гибкой разработки программного обеспечения и помогают практикам определить, как внедрить гибкую разработку в своих организациях. Мы подробно рассматриваем ситуации, в которых гибкие или традиционные методы могут быть более подходящими. Кроме того, мы рекомендуем внедрение гибридного метода, сочетающего гибкие и традиционные подходы. Гибридные методы позволяют традиционным командам разработчиков программного обеспечения внедрять некоторые практики гибкой разработки, используя сильные стороны уже существующих методов.

Статья организована следующим образом: в разделе II рассматриваются основы гибкой разработки программного обеспечения. В разделе III обсуждаются теоретические основы результатов применения гибких методологий. В разделе IV рассматриваются теоретические аспекты применения гибких методов разработки в крупных организациях. В разделе V представлена наша концепция выбора гибкой методологии разработки программного обеспечения в зависимости от условий проекта. В разделе VI рассматриваются применимые и стратегия внедрения гибких методов разработки, характерных для крупных организаций. Наконец, в разделе VII предлагаются идеи для будущих исследований, а в разделе VIII представлены выводы.

II. ОБЗОР ГИБКОЙ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Жизненные циклы разработки определяют, как группа разрабатывает программное обеспечение. Они могут определять руководителя проекта, количество участников, частоту и формальность взаимодействия в команде, основную роль и другие аспекты разработки системы. Поскольку конкретный жизненный цикл команды во многом определяет то, что происходит в процессе разработки, обзор и рекомендации по гибкой разработке в крупных организациях.

Одна из основных проблем, с которой часто сталкиваются разработчики, — это необходимость адаптации к изменениям [Остин и Девин, 2009]. Требования часто меняются после начала проекта, а заказчики и спонсоры часто меняют свои ожидания от конечного продукта. Традиционные методы разработки обычно предусматривают возможность реагирования на меняющиеся требования, но

Эти меры требуют времени и могут быть дорогостоящими. В результате многие разработчики адаптируют жизненные циклы, чтобы повысить гибкость.

Исследователи определяют гибкость по-разному, и взгляды на эту концепцию часто противостоят друг другу [Sarker et al., 2009]. В литературе по информационным системам гибкость определяется как «постоянная готовность метода [разработки информационных систем] быстро или изначально создавать изменения, проактивно или реактивно принимать изменения и учиться на них, одновременно внося вклад в воспринимаемую клиентом ценность (экономичность, качество и простота) посредством его коллективных компонентов и взаимоотношений с окружающей средой» [Conboy, 2009, стр. 377]. Эта идея создания более гибких методов приобрела широкую популярность в 1990-х годах [Austin and Devin, 2009].

В 2001 году группа разработчиков программного обеспечения обратилась, чтобы сформулировать основные принципы гибкой разработки, которые мы кратко изложили в Таблице 1. В каждом случае гибкие разработчики считают первый пункт более ценным, чем второй. Например, сторонники гибких методов ценят документацию не так высоко, как работающее программное обеспечение: «Мы ценим документацию не столько с точки зрения того, насколько она поддерживаемая и редко используемая, сколько томов» [Beck et al., 2001].

Таблица 1: Принципы Agile [Бек и др., 2001]

Принцип Agile	Описание
Сотрудничество с клиентами важнее переговоров по контракту. Сокращение формальностей для более быстрого начала и завершения работ с упором на клиента на протяжении всего процесса разработки.	
Индивидуумы и взаимодействие важнее процессов и инструментов	Улучшить коммуникацию внутри команды и снять барьеры
Рабочее программное обеспечение с черпывающей документацией	Разработчики тратят больше времени на кодирование и тестирование, чем на написание подробной документации.
Реагирование на изменения важнее следования плану	Дайте командам свободу вносить изменения и адаптироваться к потребностям проекта

В то время как некоторые разработчики стремятся интегрировать принципы Agile в существующие методологии, другие создают формализованные гибкие жизненные циклы, наиболее популярными из которых являются Scrum и Extreme Programming (XP) [Conboy, 2009]. Каждая гибкая методология использует уникальные практики, сходясь при этом к фокусу на основных принципах, представленных в таблице 1. Например, Scrum фокусируется на аспектах управления проектами в процессе разработки, используя короткие, но частые встречи команды для оценки прогресса. XP, наоборот, фокусируется на самом процессе разработки, предписывая конкретные методы, такие как парное программирование.

Методологии гибкой разработки основаны на «интенсивно итеративных процессах» [Остин и Девин, 2009, стр. 463], что означает, что команды тщательно анализируют, проектируют и пишут код в короткие интервалы времени, встречаются с заказчиком для оценки прогресса и получения обратной связи, а затем начинают цикл заново. Этот метод резко отличается от традиционных, где длительная фаза проектирования предшествует длительной фазе кодирования, которая, в свою очередь, предшествует длительной фазе тестирования.

Бэмми Тернер [2005] утверждает, что понастоящему гибкий процесс должен быть самоорганизующимся и спонтанным. Самоорганизация означает, что команды принимают решения посредством неформального общения и частых коротких совещаний, а не полагаются на одного ответственного руководителя проектом. Термин «спонтанный» в гибкой разработке означает, что требования возникают в ходе проекта, практически не тратя время на проектирование до начала кодирования.

Основное различие между гибкими и традиционными парадигмами разработки заключается в том, что гибкие парадигмы ориентированы на эффективную адаптацию меняющихся требований проекта [Остин и Девин, 2009]. Сторонники плановых методов утверждают, что время, затраченное на проектирование, играет ключевую роль в разработке качественной системы, с соответствующей спецификацией, и что снижение гибкости является необходимым компромиссом. Сторонники гибких методов, напротив, утверждают, что система, разработанная с использованием итеративного метода, по-прежнему может соответствовать всем требованиям и надлежущим стандартам проектирования, в то время как частые итерации позволяют многократно совершенствовать продукт и получать конечный продукт, более соответствующий реальным пожеланиям клиентов.

Сильные и слабые стороны

В обширной литературе подчёркиваются сильные стороны гибких методов. Во-первых, как уже упоминалось, гибкие методы обеспечивают гибкость и адаптивность [Остин и Девин, 2009]. Поскольку этап проектирования не является формализованным, а программисты работают короткими интервалами над небольшими этапами, существенные требования к проекту могут меняться в ходе разработки без существенной потери производительности.

Методы Agile также способствуют фокусировке на потребностях клиентов. Благодаря адаптивности методов Agile, запрошенные клиентами изменения в планах или требованиях часто могут быть учтены на следующей итерации разработки. Практики



обратите внимание, что гибкие методы помогают сосредоточиться на потребностях клиентов и меняющихся желаниях на протяжении всего проекта [Кокберн и Хайсмит, 2001].

Гибкие методы разработки также часто приводят к усложнению разработки, в зависимости от размера проекта и актуализации проектных рисков. Без детального проектирования или документирования на протяжении всего проекта команда может сосредоточиться на самом процессе разработки — написании кода и тестировании продукта. В небольших или средних проектах, где команды разрабатывают программное обеспечение занесколько итераций, гибкая разработка обеспечивает более короткий цикл разработки, чем методы, основанные на планировании.

Хотя гибкие методы разработки популярны и успешно применяются в небольших и средних проектах, критики быстро указывают на их недостатки. Эти недостатки часто более заметны в крупных или сложных проектах, а также в крупных, структурированных компаниях. Например, компания AG Communication Systems, имея команды разработчиков численностью от двух до нескольких сотен человек, обнаружила, что, хотя небольшие команды хорошо справляются с Scrum, гибкие методы не столь эффективны для крупных команд [Rising and Janoff, 2000].

При использовании гибких методологий разработка начинается с того, как требования будут четко определены. В сложных и/или крупных проектах такой подход может быть потенциально губительным, поскольку важные функции могут быть забыты или неправильно поняты, что потребует дополнительной работы на более поздних этапах проекта. Кроме того, без подробного плана сложно оценить время и ресурсы.

Короткие итерации гибкой разработки призваны обеспечить адаптивность и клиентоориентированность. Однако без

на этапе детального планирования достаточная оценка ресурсов и требуемого времени практически невозможна, поскольку заказчик может добавлять или удалять функции в процессе разработки. Для небольших и средних проектов такая неопределенность является приемлемым риском. Для крупных или сложных проектов величина неопределенности выше и представляет собой неприемлемый риск для большинства организаций.

В крупных проектах гибкие методы не способствуют формализации коммуникации. В одном из исследований указывалось на преимущества гибкой разработки в плане улучшения коммуникации между членами команды проекта, но также отмечалось, что при использовании гибких методов «в крупных проектах разработки, в которых задействовано множество внешних заинтересованных сторон, несоответствие адекватных механизмов коммуникации иногда может даже препятствовать коммуникации» [Pikkarainen et al., 2008, стр. 303].

Помимо снижения формального взаимодействия, скудное документирование может быть особенно губительно для крупных или сложных проектов. Программные продукты требуют периодического обслуживания, а документирование облегчает обслуживание.

С другой стороны, многие небольшие проекты требуют меньше формального обновления, чем крупные проекты, и, как таковые, не так зависят от тщательной и подробной документации.

Мы суммируем сильные и слабые стороны гибкой разработки в Таблице 2. Очевидно, что как гибкие, так и традиционные подходы имеют свои сильные и слабые стороны, и не каждый подход подходит для каждого проекта разработки.

Сильные стороны гибких методов, как правило, выгодны для небольших и менее сложных проектов, а их слабые стороны наиболее очевидны в крупных и сложных проектах. В таких крупных и сложных проектах недостатки традиционных методов помогают снизить риски, связанные с неопределенностью от структуры. Эта статья поможет принять взвешенное решение, учитывая особенности проекта, о том, какие методы лучше всего использовать: гибкие, традиционные или их комбинацию. Конечно, организации не могут полностью исключить все риски, связанные с неопределенностью проекта. Цель — снизить риски, где это возможно, и выбрать метод или методы, наилучшим образом соответствующие этой цели.

Таблица 2: Сильные и слабые стороны гибкой разработки

Сильные стороны	Слабые стороны
Ориентация на потребности клиентов	Не способствует формальному общению
Адаптируется к меняющимся требованиям	Время и ресурсы могут быть изначально неизвестны
Быстрое время разработки	Требования нечетко определены
	Отсутствует документация

Исследования гибких методологий

Ряд лучших и практиков в сообществе информационных систем обсуждали распространяемость методов гибкой разработки и последствия этого роста [например, Austin и Devin, 2009; Conboy, 2009; Sarker, 2009]. В этих статьях рассматривается и объявляется сильные и слабые стороны гибкой разработки, перечисленные выше. Многие статьи [например, Mangalaraj et al., 2009; Vidgen и Wang, 2009] посвящены чисто гибким методологиям, таким как XP и Scrum; другие статьи [например, Fitzgerald et al., 2006; Karlsson и Agerfalk, 2009; Port и Bui, 2009] обсуждают гибридные методологии, сочетание гибких и плановых методов, которые обсуждаются далее в статье. Приложение А содержит обзор литературы по гибким методологиям, а также использованные исследовательские методологии. В настоящей резюме содержатся рекомендации для будущих исследований, касающихся гибких и гибридных методологий.

Обзор литературы по гибким методологиям в крупных организациях показывает, что большинство исследований носят повествовательный характер и состоят в основном из практических примеров [Berger и Beynon-Davies, 2009; Boehm и Turner, 2005; Fitzgerald и др., 2006]. Хотя многие практические примеры дают полезные идеи и рекомендации, эти рекомендации не поддаются широкому обобщению поскольку им не хватает теоретической проработки для полного объяснения полученных результатов. Небольшое количество исследовательских статей посвящено теоретическим аспектам гибкой разработки [например, Pikkarainen и др., 2008; Sarker и др., 2009], но эти статьи не расматривают нашу основную тему, то есть гибкие методы менее успешны в крупных организациях или сложных проектах.

Одним из исключений является исследование Cao et al. [2009], которые используют теорию адаптивной структуризации для изучения успешных внедрений гибких методов путем адаптации методологий разработки. Их исследование широко использует теорию для изучения гибких методов в крупных проектах и командах. Однако наше исследование использует другой подход. В то время как Cao et al. объясняют процессы адаптации методологий к конкретным обстоятельствам, мы стремимся теоретически объяснить причины, по которым одни типы сред больше подходят для гибких методов, чем другие. Кроме того, исследуя теорию связанных зануловых успешности или неудачей внедрения гибких методов, особенно в крупных организациях или сложных проектах, мы можем создать руководящую структуру, которая поможет разработчикам сделать выбор между гибкими, основанными на плане и гибридными методологиями разработки программного обеспечения.

III. ТЕОРЕТИЧЕСКОЕ РАЗВИТИЕ

Необходимость и мотивация применения гибких методов, а также их успех или неудача могут быть лучше поняты, если рассмотреть с точки зрения организационной теории взаимозависимости и координации [Томпсон, 1967]. В своей теории организационной структуры Томпсон [1967] выделяет три типа взаимозависимости и три типа координации, используемых для управления этими взаимозависимостями. Типы взаимозависимости, присутствующие в организации, и затраты на координацию взаимозависимостей могут часitchно определять подходящий методологии разработки программного обеспечения.

Объединенные взаимозависимости возникают, когда человек свободно связан с другими людьми и разделяет общие риски. Например, клиенты страховых компаний имеют общую взаимозависимость с всеми остальными клиентами, поскольку каждый из них зависит от критической массы участников, которые могут компенсировать риск несчастного случая. Аналогично, каждый член команды разработчиков программного обеспечения имеет общую взаимозависимость с всеми остальными членами команды. Для завершения проекта каждый участник должен выполнять свою часть работы, хотя большинство участников напрямую зависят от вклада или результата каждого другого члена команды. Объединенные взаимозависимости требуют наименьших затрат на координацию. Стандартизация — это метод координации, используемый для управления объединенными взаимозависимостями. Например, чтобы гарантировать, что каждый член проектной команды выполняет свою часть работы, организация или команды устанавливают стандарты количества рабочих часов, необходимых в неделю и критерии качества результатов каждого члена команды.

Последовательные взаимозависимости возникают вследствие последовательной природы рабочего процесса. При разработке программного обеспечения анализ должен проводиться до проектирования, проектирование — до разработки, разработка — до внедрения и т.д. Другими словами, если результаты работы члена команды А требуются для В в качестве входных данных, то В имеет последовательную зависимость от А. Стоимость координации последовательных взаимозависимостей выше, чем для объединенных взаимозависимостей. Координация последовательных взаимозависимостей требует планирования, которое выполняется многократно и уникально для каждого проекта, в отличие от стандартизации, которую организациям устанавливают один раз для нескольких проектов.

Наконец, взаимные зависимости возникают, когда две или более сторон зависят друг от друга в обоих направлениях. Более того, характер зависимости может быть изначально неясен. Например, рассмотрим последовательную зависимость, в которой разработчик В требует кода, созданный А. Однако, если В обнаруживает ошибки или дефекты в коде А, код должен быть отправлен обратно А. Следовательно, существует взаимная зависимость, в которой В зависит от кода А, но А также зависит от В для тестирования и утверждения результатов А. Аналогично, рассмотрим сценарий, в котором разработчики А и В создают программные модули. Модуль В требует входных данных, которые являются одними и теми же данными модуля А. В этом случае В зависит от А для создания соответствующих входных данных, а А зависит от В для указания своих требуемых входных данных.

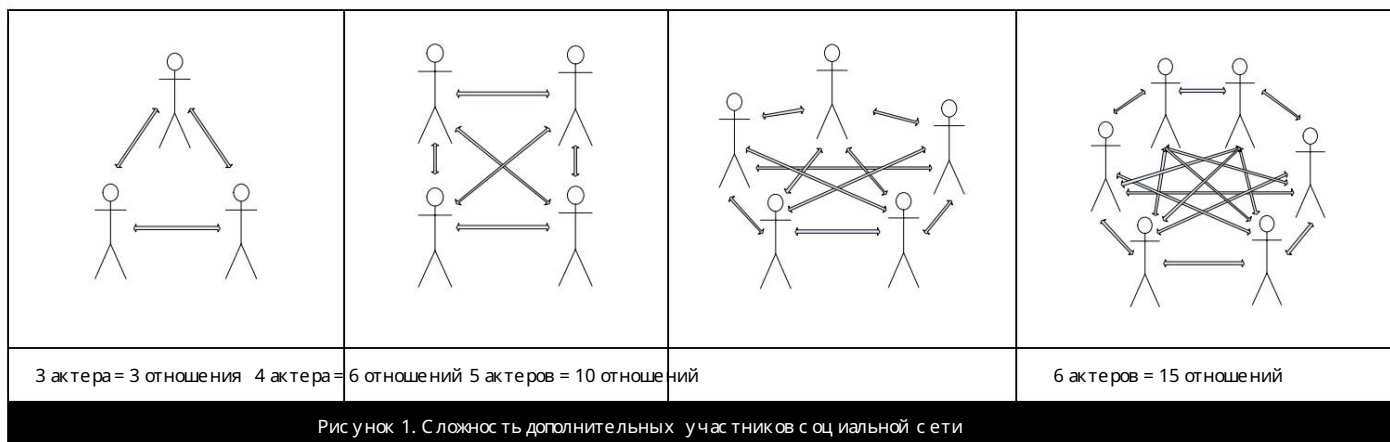
Взаимозависимость связана с высоким риском и неопределенностью проектных процессов. Например, неопределенность в процессе разработки программного обеспечения может быть результатом непредвиденных изменений в объеме работ, недостатков в планах проекта или уровня компетентности членов команды [Schmidt et al., 2001]. Взаимозависимость обычно требует координации в форме взаимного согласования, которое подразумевает неформальное общение, происходящее нерегулярно, вне рамок формальных планов проекта и часто непрерывно на протяжении всего проекта [Thompson, 1967].

Планово-ориентированные методологии предполагают, что взаимозависимости проектов в основном являются последовательными и могут управляться посредством координации в форме планирования и анализа. Однако многие взаимозависимости в жизненном цикле программного обеспечения фактически являются взаимобусловленными. В результате часть времени и средств, затрачиваемых на создание подробных планов, тратится впустую и требуется определенная степень взаимной корректировки. В отличие от них, гибкие методологии предполагают противоположное. Они рассматривают большинство или все взаимозависимости проектов как взаимобусловленные и, следовательно, используют взаимобусловленность

координируют все взаимозависимости проекта. Другими словами, они снижают важность формального предварительного планирования и координируют действия по мере возникновения необходимости. Однако, согласно структурной теории непредвиденных обстоятельств [Donaldson, 2001; Thompson, 1967], ненужные затраты на использование взаимной корректировки вместо планирования последовательных взаимозависимостей являются пустой тратой времени. В идеале, команды ИТ-проектов должны использовать методологию, основанную на гибридной стратегии координации, которая использует взаимную корректировку только для взаимных взаимозависимостей и планирование для последовательных взаимозависимостей. Соответственно, наши рекомендации основаны на этом теоретически идеальном сценарии.

Помимо использования подходов стратегии координации взаимозависимостей, существующих в процессе разработки программного обеспечения, проектные команды должны иметь инфраструктуру для поддержки выбранной ими метода координации. Например, для эффективного использования планирования руководителям проектов необходимо полностью понимать характер всех последовательных взаимозависимостей, объем требуемых отдельных задач и ресурсы, необходимые для их выполнения. Кроме того, они должны уметь точно оценивать временные и финансовые требования, что требует значительных знаний и опыта от разработчиков планов. Без этих необходимых знаний необходима система, помогающая новым руководителям повторно использовать знания и опыт, накопленные в ходе предыдущих проектов.

Напротив, для эффективного использования взаимной адаптации посредством неформального обмена знаниями и информацией, сильная, хорошо развитая социальная сеть должна обеспечивать неформальное общение. Таким образом, организациям следует минимизировать затраты на коммуникацию, обеспечивая такие условия, как совместное размещение членов команды или наличие подходов к редств коммуникации. Это требование отчасти объясняет, почему гибкие методы сложно применять в больших группах: количество дополнительных связей в социальных сетях, которые необходимо создать для адекватного включения члена команды, экспоненциально возрастает с каждым добавленным в сеть субъектом, как показано на рисунке 1. В действительности большинству субъектов не потребуется взаимодействовать друг с другом. Однако без четко определенных и предсказуемых ролей, областей действия и каналов коммуникации каждому субъекту потенциально может потребоваться взаимодействовать с любым другим субъектом.



Этих теоретических выводов недостаточно для определения большинства управленческих действий. Руководители проектов могут предположить, что низкая производительность проектов разработки программного обеспечения является результатом несоответствия выбранной методологии взаимозависимости проекта. Например, в плановой среде низкая производительность может быть следствием не слишком сильной взаимной взаимозависимости, а скорее неразвитой инфраструктуры для поддержки планирования проекта. В этой ситуации наилучшим решением будет улучшение инфраструктуры планирования, а не переход на гибридную методологию. Аналогично, низкая производительность в гибкой среде может быть результатом слабой социальной сети, а не наличия последовательных взаимозависимостей. Таким образом, принятие оптимального решения относительно методологии разработки программного обеспечения требует понимания как характера взаимозависимостей процесса, так и способности поддерживать необходимый для этой взаимозависимости тип координации.

Теория обработки информации [Гэлбрейт, 1973; Ташман и Надлер, 1978] утверждает, что эффективность деятельности компании основана на достижении «соответствия» между её потребностями в обработке информации и её возможностями. Неопределенность и взаимозависимость, связанные с основными бизнес-процессами, порождают потребность в обработке информации.

Возможности организации по обработке информации обусловлены различными источниками, включая её информационные технологии, формальную организационную структуру и способность поддерживать неформальную координацию.

Если потребности организации в обработке информации невелики, её информационные системы и формальная структура должны быть разработаны таким образом, чтобы поддерживать координацию посредством планирования и стандартизации. Если потребности в обработке информации высоки, руководителю следует разработать информационные системы и формальную структуру, которые будут поддерживать неформальное общение и сотрудничество, а также иметь достаточно свободных ресурсов. Несовпадение между информацией организации и

Потребности в обработке информации и связанные с ней структура поддержки приводят либо к низкой производительности обработки информации, либо к напрасной трате ресурсов (см. Таблицу 3).

Таблица 3: Перспектива обработки информации [Гэлбрейт, 1973; Ташман и Надлер, 1978]			
		Потребности в обработке информации	
		Высокий	Низкий
Возможности обработки информации	Высокий	Оптимальная производительность	Потраченные впустую ресурсы
	Низкий	Низкая производительность	Оптимальная производительность

Другая потенциальная ошибка — предполагать, что природа взаимозависимостей проекта детерминирована и неизменна. Взаимозависимости возникают из-за неопределенности [Томпсон, 1967]. Таким образом, изменения неопределенности и риска внутри организации приводят к изменениям во взаимозависимостях. Используя пример взаимной взаимозависимости, приведенный выше, разработчик А взаимозависим с В из-за неопределенности относительно того, будет ли программный модуль работать так, как планировалось, без пользовательского тестирования. Неопределенность возрастает, если разработчики неопытны, менее

Знающие или не привыкшие к совместной работе. Этот тип риска становится еще более опасным в крупных организациях с высокой текучестью кадров.

Организации могут минимизировать риск, оптимизируя формальную организационную структуру, создавая автономные, модульные задачи [Galbraith, 1973; Thompson, 1967] или выбирая средства коммуникации, соответствующие задаче [Dennis et al., 2008]. Например, менеджеры должны формально группировать роли и лиц, которые естественным образом имеют наибольшую взаимную взаимозависимость, чтобы снизить затраты на их координацию [Thompson, 1967]. Если менеджеры также успешно модуляризуют процесс разработки программного обеспечения в независимые, автономные задачи, меньше взаимных взаимозависимостей возникнет между группами разработчиков. Наконец, если менеджеры правильно группируют роли и модуляризуют задачи, они могут выбрать соответствующую информационную поддержку и взаимной координации внутри групп и последовательной координации между группами. Организации, которые успешно реализуют эти стратегии, могут быть в состоянии создать предпочтительные альтернативы для методологии разработки программного обеспечения и поддерживающей инфраструктуры.

IV. РАЗРАБОТКА ПО МЕТОДАМ AGILE В ЗРЕЛЫХ ОРГАНИЗАЦИЯХ

Большинство статей, демонстрирующих успешность гибких жизненных циклов, используют в качестве доказательств реальные сценарии, но эти сценарии, как правило, относятся к небольшим или низкорисковым проектам, часто в небольших организациях [например, Кокберн и Хэйсмит, 2001; Куссмаул и др., 2004; Пиккарайнен и др., 2008; Райзинг и Янофф, 2000]. Небольшие проекты, описанные в этих статьях, позволяли командам поддерживать инфраструктуру, которая соответствует взаимозависимостям проекта. Например, в одном случае, изученном Пиккарайненом и др. [2008], команда проекта состояла всего из шести человек. Задачи членов команды по созданию системы управления безопасностью были взаимно взаимозависимыми, что требовало постоянного неформального общения между ними. Из-за небольшого размера команды гибкие стратегии, такие как открытые офисные пространства и неформальные встречи команды, успешно поддерживали потребности проекта. Таким образом, гибкая разработка лучше всего подходит для небольших или средних совместных распределенных команд [Бем и Тернер, 2003; Линдвалл и др., 2004].

Напротив, крупные или сложные проекты и/или зрелые организации представляют собой более проблемные сценарии для внедрения гибких методологий. Поскольку крупные, сложные проекты и организации, естественно, имеют множество взаимозависимостей, проекты требуют значительного объема координации. В результате менее затратные формы координации — стандартизация и планирование — легче контролировать в таких ситуациях. Другими словами, большинство людей, работающих в крупных проектах и организациях, сложно реализовать взаимную координацию больших масштабов. Поскольку гибкие методы основаны на взаимной адаптации, эти методы обычно не подходят для крупных, сложных проектов или зрелых организаций.

Сложность, характерная для зрелых организаций, связана с такими фреймворками управления ИТ, как ITIL, CMMI или COBIT.1 Эти фреймворки обеспечивают согласованность ИТ с бизнес-целями и структурируют процессы разработки и управления ИТ. В типичной гибкой среде структура, установленная фреймворком управления, может препятствовать реализации проекта [Boehm and Turner, 2005]. Мы рассмотрим эту проблему далее в статье, предложив использовать гибридные методологии.

Исследователи и практики, изучающие или внедряющие гибкие методы в крупных, зрелых организациях, отмечают связанные с этим трудности [например, Бергери и Бейнон-Дэвис, 2009; Бем и Тернер, 2005; Линдвалл и др., 2004; Пиккарайнен и др., 2008; Пиккарайнен и Пасоя, 2005]. Сторонники гибких методологий предлагают масштабировать их на более крупные команды и проекты [например, Кон, 2007], но в нашем обзоре научных и практических журналов мы не нашли ни одной статьи с подробными историями успеха гибких жизненных циклов в крупных организациях (см. Приложение А).

¹ Определения аббревиатур: Библиотека инфраструктуры информационных технологий (ITIL), Интегриция модели зрелости возможностей (CMMI) и Цели управления для информации и связанных с ней технологий (COBIT).

Очевидно, что гибкая разработка в её самой строгой форме, вероятно, не является хорошим решением для многих инициатив в области разработки в крупных, зрелых организациях. Как следует из наших рассуждений, команды и/или проекты в отмеченных выше случаях были слишком велики, чтобы полностью поддерживать неформальный характер методов гибкой разработки. Хотя эти проекты могли подразумевать взаимные зависимости, инфраструктура была несовместима с чисто гибкими методологиями. В тех случаях, когда взаимные зависимости присутствуют, но гибкие методологии нереализуемы, гибридная методология, которую мы обсудим в следующих двух разделах, может быть оптимальным решением.

V. ВЫБОР МЕТОДОЛОГИИ

На основе обзора литературы и теоретических наблюдений затруднения применения гибких методов в крупных организациях мы представляем структуру выбора методологии, соответствующей потребностям организаций. Наша структура представлена на рисунке 2, где проиллюстрированы рекомендуемая методология, стратегия координации и варианты инвестиций для поддержки координации проектных команд разного размера, взаимозависимости и волатильности. На рисунке 2 мы используем термин «волатильность» для обозначения нестабильности, связанной с текучестью кадров в проектной команде. Текучесть кадров становится более вероятной при резких изменениях в экономике. В периоды бурного роста компании нанимают больше сотрудников, которым нужно время для адаптации к корпоративной культуре и развития соответствующих навыков. Во время рецессий многие организации увольняют высокооплачиваемых сотрудников в пользу выпускников вузов, которым они могут платить меньшую зарплату. Поэтому мы различаем нашу теоретическую структуру для среднесрочной и низкой волатильности команд.

Когда большинство взаимозависимостей проекта являются последовательными (Рисунок 2, Вставки A и D), независимо от размера команды или волатильности проекта, руководитель проекта должен принять методологию основанную на плане, и инвестировать в технологии, которые будут поддерживать процесс планирования проекта. Например, будут полезны системы управления знаниями, которые могут облегчить хранение и поиск прошлых планов проекта, ресурсов, ключевых показателей эффективности, успехов, неудач и т. д. Новые руководители проектов могут войти в игру, используя документированную информацию и ожидать, что она будет относительно точным индикатором будущих результатов. Опыт и обучение менее важны в этой среде, поскольку руководители проектов могут легко квалифицировать соответствующие знания по проекту в системе управления знаниями. Связь в социальных сетях, которая способствует неформальному обмену знаниями, также менее важна, поскольку большинство необходимых каналов коммуникации можно предсказать, определить и спланировать заранее. В результате размер команды менее важен, поскольку координация происходит в ходе планирования проекта, а не в ходе реализации через неформальные социальные сети.

Аналогично, формальные организационные структуры не обязательно должны обновляться на группировке взаимных взаимозависимостей и могут следовать друг другу стратегиям, например, объединять сотрудников с высоким и низким опытом для стимулирования создания и передачи знаний [Nonaka, 1994] или объединять сотрудников с разными ролями для стимулирования экономии за счёт масштаба [Panzar and Willig, 1981]. Единственное отличие в случае, когда большинство взаимозависимостей проектов являются последовательными, заключается в том, что организации с низкой изменчивостью состава команды (рисунок 2, вставка D) также могут рассмотреть возможность инвестирования в компетентность и неявные знания своих руководителей проектов. При меньшей текучести кадров среди руководителей проектов организации с большей вероятностью могут воспользоваться преимуществами инвестиций в развитие неявных знаний.

Когда взаимозависимости проекта имеют обратный характер (рисунок 2, блоки B, C, E и F), наши теоретические рекомендации варьируются в зависимости от размера и волатильности команды проекта. Если размер команды проекта небольшой (блоки C и F), мы рекомендуем использовать гибкую методологию. Гибкие методы с итеративными циклами и частым взаимодействием между членами команды и заинтересованными сторонами хорошо подходят для небольших команд с высокой степенью взаимной зависимости.

Однако подход, который следует использовать руководителям проектов для поддержки частого общения, варьируется в зависимости от нестабильности команды. Если нестабильность низкая (вставка F), руководителям проектов следует развивать сильную социальную сеть для неформального обмена знаниями, минимизируя деструктивные изменения в структуре сети. Например, анализируя личности, демографические характеристики и ценности членов команды [Klein et al., 2004], руководители проектов могут объединять людей, наиболее склонных к развитию дружеских отношений и обмену знаниями. Аналогичным образом, руководители проектов могут улучшить взаимодействие в социальных сетях, создавая группы, в которых участники выполняют схожие задачи, или проводя обучение с соответствующим технологическим совместной работы [Keith et al., 2010].

Если волатильность высока (рисунок 2, вставка C), усилия по улучшению социальных сетей могут оказаться непродуктивными из-за текучести кадров. Следовательно, лучшей стратегией для руководителя проекта по координации взаимозависимостей будет группировка ролей, которые с наибольшей вероятностью будут взаимозависимыми [Томпсон, 1967]. В этом случае любой, кто присоединится к команде для выполнения этих ролей, уже будет находиться в непосредственной близости и будет связан формальной организационной структурой.

Наконец, мы рекомендуем организациям с преимущественно взаимной взаимозависимостью и большими размерами групп использовать гибридную методологию (рисунок 2, вставки B и E). Как будет обсуждаться в следующем разделе, гибридная методология требует от руководителей декомпозиции задач проекта на максимально независимые модули. После модуляризации проекта руководитель может использовать методы, основанные на планировании, для гибких модулей проекта, имеющих преимущественно последовательную взаимозависимость, и гибкие методы для большинства модулей, имеющих взаимную взаимозависимость. Если такие проекты успешно модуляризируются, руководители проектов могут использовать методы, основанные на планировании, для координации действий подгрупп.

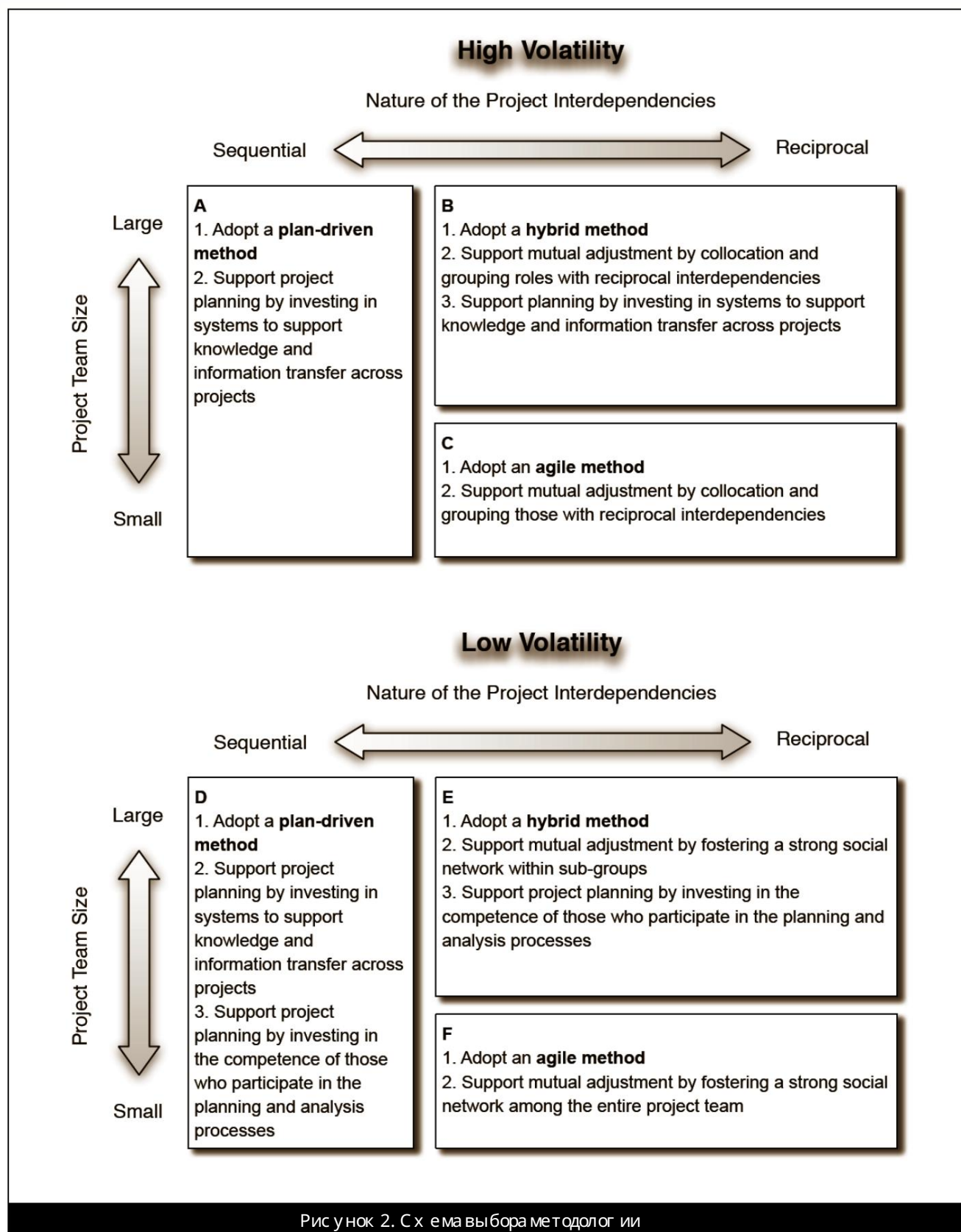


Рисунок 2. Схема выбора методологии

Например, парадигмы сервис-ориентированности и облачных вычислений помогают разработчикам создавать ИТ-системы из слабозавязанных, независимых программных модулей или сервисов [Bardhan et al., 2010]. Если организация технически может спроектировать новую систему таким образом, руководитель проекта может естественным образом разложить процесс разработки программного обеспечения на составляющие, основанные на этих модулях. Руководителям проектов не нужно заботиться о том, разрабатывается ли каждый модуль с использованием плановой или гибкой методологии, — важно лишь, чтобы каждый программный модуль принимал необходимые входные данные и производил требуемые результаты, соблюдая стандарты качества.

Хотя мы рекомендуем гибридный подход для крупных организаций с взаимными взаимозависимостями, подходы инфраструктуры для поддержки координации проекта зависят от уровня волатильности команды. При низкой текучести кадров (вставка E) организации могут поощрять связь в социальных сетях с большей уверенностью в том, что их усилия не будут напрасными. По той же причине эти организации также должны инвестировать в развитие новых знаний с своих руководителей проектов и общей компетентности. Организации должны иметь как хорошо сбалансированную консультативную сеть, так и иллюзорную транзакционную память среди членов команды. Транзакционная память относится к тому, насколько хорошо каждый член команды проекта понимает сильные стороны и опыт других членов команды [Faraj and Lee, 2000]. Кроме того, команды проектов должны иметь возможность и структуру поддержки для эффективного обмена этими знаниями и опытом во времени и пространстве, поскольку становится все труднее размещать всю команду проекта по мере ее роста.

Напротив, при высокой волатильности (вставка B) инвестировать в социальные сети, которые легко разрушаются, или в часто меняющихся менеджеров проектов могут оказаться нецелесообразными. Вместо этого от организации следует сосредоточиться на систематизации как можно большего объема знаний в системе управления знаниями, которая поможет в планировании проектов и поиске поставщиков. Этим организациям также следует сгруппировать роли, которые с наибольшей вероятностью будут взаимозависимыми.

Важно отметить, что основное внимание уделяется группировке ролей, а не отдельным лицам, поскольку фактические сотрудники, исполняющие эти роли, часто меняются.

Мы обновляем нашу концепцию ее применение на практике на соответствующих теоретических основах, касающихся организаций, структуры и технологий [Galbraith, 1973; Thompson, 1967]. Ключевым моментом является достижение соответствия между взаимозависимостью размером и изменчивостью команды, выбранной методологией разработки программного обеспечения и поддерживающей инфраструктурой. Многие важные факторы, выявленные в предыдущих исследованиях ИТ-проектов, такие как риск и координация [Kraut and Streeter, 1995; Schmidt et al., 2001], также основаны на этих базовых концепциях.

Хотя наши предложения основаны на теории, они, безусловно, могут различаться в зависимости от реальной стоимости трех вариантов. Например, Фицджеральд и др. [2006] приводят пример успешного внедрения гибридного метода. Команды Intel Shannon в данном случае были относительно небольшими, от четырех до шести человек. Наша концепция предполагает, что гибридные методы наиболее подходят для более крупных команд, а меньшие команды больше подходят для более строгих форм гибкой разработки. Однако, подобно более крупным командам и проектам, Intel Shannon, по-видимому, ставит с вою потребность в структуре, формальной коммуникации и соблюдении принципов управления выше преимуществ, обещанных гибкими методологиями. Организация внедрила лишь части гибких методологий, с которыми многие стратегии снижения рисков, заложенные в ее традиционных методах. Мы стремимся к тому, чтобы наша концепция и рекомендации представляли собой набор руководящих принципов, которые можно адаптировать к конкретным ситуациям в соответствии с конкретными потребностями организации.

VI. ГИБРИДНЫЕ МЕТОДОЛОГИИ В КРУПНЫХ ОРГАНИЗАЦИЯХ

Поскольку чисто гибкая методология не подходит для общего использования в крупной, зрелой организации, мы рекомендуем, где это уместно, внедрять гибридное традиционное/гибкое решение, которое позволит проектным группам воспользоваться зрелостью организации в разработке программного обеспечения и одновременно получить преимущества гибкой разработки, такие как адаптируемость к изменяющимся требованиям.

Поддержка сочетания гибких и традиционных методов разработки продолжает расти в научной литературе [Fitzgerald et al., 2006]. Гибридные решения часто оказываются более успешными, чем другие методы в крупных организациях [Boehm and Turner, 2003; Cao et al., 2004; Cao et al., 2009]. Внедряя гибридные традиционные и гибкие методы, крупные организации могут воспользоваться некоторыми преимуществами гибкой разработки, не отказываясь от стабильности, обеспечиваемой традиционными методами.

Некоторые гибкие методологии являются успешной альтернативой традиционным практикам разработки при использовании в ситуациях, описанных нашей платформой и теорией. Наши рекомендации основаны, главным образом, на опубликованных результатах реального опыта различных подразделений разработки программного обеспечения. Несколько исследований [Harris et al., 2009; Maruping et al., 2009a; Maruping et al., 2009b] показывают высокую эффективность гибких методов, особенно при изменении требований в ходе проекта. Как упоминалось ранее, некоторые успешные внедрения конкретных гибких методов включают проекты в крупных организациях.

Примерами крупных компаний, успешно применяющих гибкие практики в гибридных жизненных циклах крупных проектов, являются Nokia [Kahkonen, 2004] и Motorola [Bowers et al., 2002]. Kahkonen [2004] подробно описывает три стиля разработки, используемых в Nokia в крупных проектах. Философия Nokia заключается в том, что крупные проекты требуют большого количества участников команды, но коммуникация и координация становятся все сложнее по мере роста команд. Случай Kahkonen демонстрирует способы, которыми крупная организация внедряла только части XP, чтобы смягчить некоторые негативные последствия такого увеличения усилий по коммуникации и координации. Это полностью соответствует нашей теоретической базе.

рекомендаций, в которых мы предлагаем гибридную методологию для проектов или команд, которые являются большими или сложными, но для которых все еще желательны некоторые преимущества гибкой методологии.

Большие команды разработчиков в высоко структурированном отделе Motorola внедрили гибридный жизненный цикл, используя некоторые практики методологии XP [Bowers et al., 2002].

Этот успешный проект состоял из пятнадцати различных взаимозависимых частей, что указывает на необходимость взаимной координации и корректировки в формах гибких практик.

По словам сотрудника Motorola, «мы взяли XP, перенесли одни практики, отказались от других и дополнили третьи практиками из [традиционных методов разработки]» (стр. 100). Для поддержки принятых практик XP организация распределила сотрудников по местам, переставив рабочие места для более удобного парного программирования. Эти изменения в инфраструктуре и методологии помогли Motorola добиться успехов в проекте.

Как отмечалось ранее, системы управления ИТ, применяемые в крупных организациях, могут конфликтовать с некоторыми гибкими методологиями. Таким образом, организациям следует использовать лишь несколько гибких методов, а не весь жизненный цикл. Например, организация такого типа может получить наибольшую выгоду, отказавшись от принципа «отсутствия ответственности» и внедрив другие гибкие методы, такие как частые итерации [Bowers et al., 2002]. Такая стратегия позволяет сохранить существующую структуру, в то время как другие методы способствуют достижению гибкости проекта.

Одной из областей, особенно затронутых изменениями в методологиях разработки, является управление релизами. Управление релизами является одним из основных компонентов ITIL, и принципы управления релизами играют важную роль и в других фреймворках. Поскольку управление релизами обычно сосредоточено на внедрении нового или обновленного программного обеспечения, работе с клиентами над плановыми релизами и тестировании качества, гибкие методологии меняют функции команд управления релизами. Необходимо больше времени на взаимодействие с клиентами, поскольку гибкие методологии требуют регулярной обратной связи от клиентов. Кроме того, тестирование качества проводится чаще.

Некоторые организации уже достигли зрелости процессов в рамках системы управления, одновременно используя некоторые гибкие методы в своих циклах разработки. Например, Intel Shannon [Fitzgerald et al., 2006] получила оценку 2-го уровня по модели зрелости возможностей, что подразумевает определенную дисциплину и структурированность процессов разработки. Компания столкнулась с значительным ростом численности персонала (т.е. с высокой волатильностью).

Анализ требований и фактическая разработка требовали постоянного взаимодействия между группами, что подразумевало взаимную взаимозависимость проектов. Как следует из нашей модели, руководство Intel Shannon использовало гибридный метод, реализуя различные элементы методологий Scrum и XP. Руководители отказались от других компонентов этих стратегий, чтобы создать индивидуальное сочетание гибких методов, подходящих для их организационной структуры, — стратегию, которую они назвали «инжиниринг методов».

Разработка гибридных методологий

Как мы уже отмечали, методологии разработки могут включать в себя элементы гибких методологий различными способами. Поскольку каждая организация и проект отличаются друг от друга, далее мы рассмотрим и опишем общие методы разработки гибридных методологий. Организациям следует с начала оценить размер проекта, его волатильность и взаимозависимость, чтобы определить, подходит ли им гибридный метод. В этом случае они могут выбрать один или несколько из следующих методов для разработки с помощью методологии.

Адаптированная базовая методология

Карлсон и Агерафальк [2009] предполагают, что организации «стремятся разработать метод, который соответствует ситуации и в то же время согласуется с основными целями и ценностями метода» (стр. 301). Команды выбирают базовую методологию, а затем оценивают каждый компонент, чтобы определить, соответствует ли его назначение целям организации или проекта.

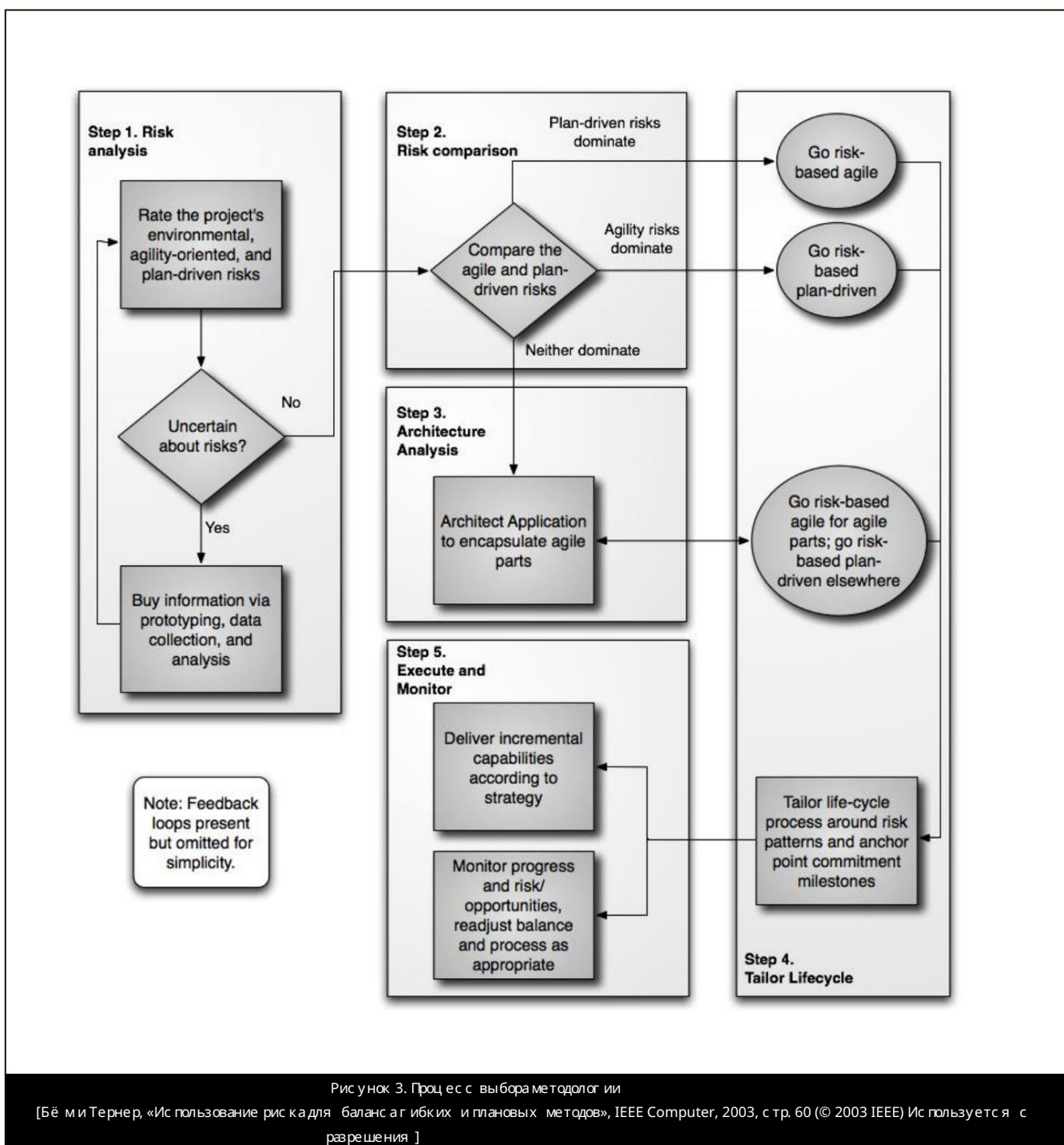
Пиккард и Пасоа [2005] предлагают аналогичный процесс. Исходя из нового внимания уделяется целям организации и тому, насколько гибкие методы им соответствуют. Команды внедряют компоненты, которые считаются эффективными, и отбрасывают неподходящие.

Методология, основанная на оценке риска

Бём и Тернер [2003] предлагают гибридную методологию основанную на оценке рисков (см. рис. 3). Согласно этой методологии, организации анализируют риски, присутствующие в проекте, и соответствующим образом адаптируют свой жизненный цикл, используя предложенную модель для более полного удовлетворения потребностей. Позднее Бём [2010] расширил эту концепцию включив в неё модель постепенных обязательств, которая позволяет организации непрерывно отсчитывать риски в проектах и устанавливает определённые точки, в которых проект может быть прекращён, если риск становится слишком высоким.

² Избирательно применяемые практики XP включают сокращение сроков выпуска, непрерывную интеграцию парное программирование, простоту проектирования, планирование, ориентированное на пользователя, рефакторинг и непрерывное тестирование. Описание каждой из этих стратегий XP см. в работе Бека и Андреса, 2004.





Анализ затрат и выгоды

Остин и Девин [2009] предлагают использовать сравнение выгоды и затрат конкретных гибких практик и внедрять только те гибкие принципы, которые в целом обеспечивают большую выгоду, чем затраты. Используя процесс, аналогичный методологии, основанной на оценке рисков, организация оценивает список возможных гибких методов и суммирует чистую выгоду от их применения в каждом конкретном случае.

Принципы Agile применяются только на части жизненного цикла

Другой популярный гибридный метод, который рассматривают организации, — это использование отдельных практик из гибких методологий при сохранении общего плана жизненного цикла разработки. Например, руководитель может выбрать гибкие методы, такие как принятие решений командой и частые итерации на этапах кодирования и тестирования [Ambler, 2008].

Проекты по-прежнему будут включать в себя полную фазу проектирования, при этом программисты смогут пользоваться такими преимуществами, как частая обратная связь и возможность принимать решения на этапах кодирования и тестирования.

Некоторые практики гибкой разработки, такие как парное программирование, можно внедрить без прерывания текущих процессов. Исследования показывают, что парное программирование улучшает производительность [Bowers et al., 2002] и снижает плотность дефектов кода [Fitzgerald et al., 2006]. Однако руководителю следует избегать некоторых гибких практик, таких как сокращение документации, которая не соответствует размеру и культуре организации [Selic, 2009]. В таблице 4 представлен список гибридных практик, особенно подходящих для крупных проектов и организаций. Мы адаптировали таблицу из исследования Као и др. [2004], которые изучали успешно реализованный гибридный подход в крупной организации. Мы рекомендуем крупным, зрелым организациям начинать с рассмотрения этих практик при разработке гибридного решения.

используя методы, которые мы упоминали ранее.

Таблица 4: Гибридные методы для сложных крупномасштабных проектов [Cao et al., 2004]	
Гибридные практики, подходящие для крупных организаций и проектов	Описание
Проектирование заранее	Хотя гибкие методологии обычно исключают предварительное проектирование, крупные или сложные проекты не могут обойтись без предварительного проектирования. Для небольших проектов этот вопрос может быть не столь важен.
Короткие циклы выпуска с многоуровневым подходом	Независимо от размера проекта или организации, полезно иметь работающее программное обеспечение в конце каждого цикла, чтобы быть готовым к тестированию и обратной связи.
Суррогатное взаимодействие с клиентами	Из-за сложности получения постоянной обратной связи от всех заинтересованных клиентов в крупных и сложных проектах, менеджеры по продуктам должны иметь прямой контакт с клиентами для постоянной обратной связи по проектам.
Гибкое парное программирование	Парное программирование успешно применяется в самых разных проектах и организациях. Као и др. [2004] рекомендуют «гибкое» парное программирование, то есть разработчики могут использовать его как можно чаще, но должны быть достаточно гибкими, чтобы понимать, что оно не будет работать во всех ситуациях.
Выявление и управление разработчиками	Нанимайте и используйте разработчиков, которые более способны работать в среде гибкой разработки. Если вы используете гибридные методологии только в некоторых проектах, обязательно назначайте подходящих разработчиков.
Повторное использование с рефакторингом	Повторное использование существующего кода для создания новых функций. Хотя отсутствие документации в гибких методологиях затрудняет повторное использование, рефакторинг (очистка кода для облегчения его адаптации к новым проектам) может помочь.
Более плоские иерархии с контролируемым делегированием полномочий	Предоставление разработчикам возможности принимать важные решения в коде ускоряет разработку, а короткие циклы помогают исправлять любые проблемы, которые могут возникнуть из-за такой практики.

Разработка сервисно-ориентированного программного обеспечения

Другой гибридный метод, называемый сервисно-ориентированной разработкой программного обеспечения (SOSD), заключается в разделении работы над конкретным проектом на отдельные компоненты, называемые сервисами [Keith et al., 2009]. Название метода происходит от сервисно-ориентированной архитектуры (SOA), в которой разработчики создают приложения из набора базовых данных, независимых программных сервисов. В методологии SOSD подгруппы внутри общей команды проекта выступают в качестве поставщиков услуг, выполняющих независимые задачи в процессе разработки программного обеспечения. Интерфейсы между сервисами или видами деятельности явно определены, но поставщикам одного сервиса не требуется понимать внутреннюю работу любого другого сервиса. В результате подгруппы могут выполнять каждую услугу, требуемую общим планом проекта, используя собственную уникальную методологию, будь то основанную на плане или гибкую.

Например, типичный проект включает фазы проектирования, кодирования, тестирования и развертывания. Команды могут разделить каждую фазу на отдельные услуги, которые будут выполняться отдельными специалистами или небольшими группами. Внутри этих услуг существуют подслужбы, обеспечивающие определенную функциональность. Одним из типов подслужб для фазы разработки может быть разработка приложения с компонентом базы данных. Другим типом подслужбы может быть разработка компонента приложения без базы данных.

На этапе планирования проекта команда может выбрать и задокументировать необходимые услуги. Затем менеджеры проектов могут сопоставлять доступные ресурсы организации с услугами проекта. Таким образом, планировщики проектов могут легко увидеть ресурсы, доступные для удовлетворения их потребностей. Хотя общий процесс координации поставщиков услуг индивидуальные усилия формальны и планируются, каждая уникальная услуга может быть выполнена с использованием методологии по выбору поставщика услуг, включая гибкие методы.

³ Один из примеров, основанный на работе Турки и соавторов (Turk et al., 2005), заключается в том, что членами гибкой команды должны быть те люди, чьи личные качества, демографические показатели, навыки и т. д. позволяют им справиться с задачами или встроенными в неформальную консультативную сеть гибкой команды.

Подводя итог, можно сказать, что расматриваемые нами гибридные подходы представляют собой альтернативу для организационных, чьи проекты зачастую имеют слабые функции традиционных методов, с низкими рисками, сочетаются с итеративными, клиентоориентированными и более гибкими функциями гибких методов, что позволяет раскрыть сильные стороны обоих подходов. Однако при слиянии подходов некоторые риски, которые в противном случае снижались бы традиционными методами, могут проявиться легче. Организациям следует тщательно выбирать оптимальный подход, обеспечивающий баланс между преимуществами традиционных и гибких подходов, учитывая их толерантность к риску и цели проекта.

Рекомендации по внедрению

Помимо выбора или разработки правильной гибридной методологии, существенное внимание требует её внедрение. Как и в случае любых других организационных изменений в политике или процессах, для перехода от плановых подходов к гибким или даже гибридным методологиям необходимы продуманные методы управления изменениями. Что касается внедрения гибридных методологий разработки, у нас есть несколько конкретных рекомендаций.

Основываясь на нашей теоретической базе и обзоре соответствующей литературы, мы с начала выделяем два основных требования для внедрения гибридного решения. Во-первых, руководители проектов должны осознавать уровень и различия между взаимозависимостями между задачами проекта. Это позволяет сделать соответствующий выбор между гибкими и плановыми методологиями для координации взаимозависимостей для каждого модуля процесса разработки программного обеспечения. Во-вторых, чтобы сделать первую очередь возможной, руководители должны точно модулизовать процесс проекта на набор мелких зернистых независимых задач. Команды должны разделить эти задачи по естественным контрольным точкам, которые также точно разделяют задачи с высоким и низким риском и задачи с взаимной и последовательной взаимозависимостью. Из нашего обзора существующих методов эти цели могут быть достигнуты, например, путем принятия модели, основанной на рисках, Бёма [2010] для первой и фреймворка SOSD Кейта и др. [2009] для второй.

Кроме того, организации могут использовать пилотные тесты для определения эффективности гибридных методов в организации с минимальным риском. Разработчикам следует использовать пилотные тесты на небольших проектах, специально отобранных в соответствии с критериями гибких методов, как показано в структуре. Выбор орошо подходов проектов будет способствовать успеху проекта и обеспечит максимальную эффективность гибридных методов. Эти пилотные тесты дают представление о том, какие практики работают, а какие нет. Кроме того, команды могут применять опыт, полученный в ходе пилотных тестов, когда более крупные подразделения организации внедряют гибкие методы. Кроме того, организациям необходимо изменить некоторые бизнес-процессы, чтобы они соответствовали процессу гибкой разработки. Например, традиционные оценки производительности неэффективны, когда программирование выполняется парами [Bowers et al., 2002]. Организациям следует адаптировать политики и процессы во время пилотного тестирования для оценки эффективности изменений. Кроме того, менеджерам следует создавать команды с разным уровнем опыта и знаний. Однако мы не рекомендуем включать в пилотные команды новичков в программировании [Boehm and Turner, 2005].

Далее организациям следует определить конкретные обязанности или типы проектов, к которым будут применяться гибкие методы. Бёма и Тернер [2005] предлагают использовать «небольшие приложения с интенсивным графическим интерфейсом и коротким жизненным циклом» (стр. 32) для начала экспериментов с гибкими методами в организации. Ограничивая применение гибких методов определенным типом проектов, определенных в нашей структуре, гибкие методы имеют больше шансов на успех.

Переход от плановых к гибким гибридным методологиям отнюдь не тривиален. Как показывают наша теория и обсуждение, наличие в организации правильных механизмов, позволяющих учитывать изменчивость и взаимозависимость ситуаций, имеет ключевое значение. Эти изменения затронут все этапы процесса разработки, от руководителей проектов и разработчиков до заказчиков. Как и в случае любых изменений, появятся противники. Некоторые могут опасаться работать в гибкой среде. Одно из преимуществ внедрения гибридных методов в отдельных проектах заключается в том, что проекты, использующие преимущественно традиционные методы, с которыми это преимущество должно помочь справиться, чтобы не вызывать негативных эмоций при внедрении изменений.

Все подразделения организации, участвующие в пилотных проектах, должны быть осведомлены об изменениях, которые гибкие практики привнесут в организацию. Помочь людям принять изменения может быть сложнее, чем реализовать их [Кокберн и Хэйс-мид, 2001]. Клиенты должны быть более доступными для консультаций с проектными командами. Разработчики должны быть более гибкими и готовыми принимать меняющиеся требования пользователей. Руководители проектов должны адаптироваться к новой роли при управлении гибкой командой. Роль руководителя проекта может измениться с принятием решений на одобрение их принятия [МакЭвой и Батлер, 2009]. В некоторых случаях культура может быть настолько укоренена в проектной команде, что некоторым командам может быть выгоднее использовать методологию, которая не обязательно соответствует их взаимозависимостям и неопределенностям.

Например, командам разработчиков, привыкшим к четко определенным ролям и политикам, особенно в области принятия решений, следует в первую очередь прибегать к более традиционным методам, если культура такова, что члены команды не могут адаптироваться к изменениям [Boehm and Turner, 2003]. Эти крайние случаи являются исключениями из представленной нами модели.

VII. БУДУЩЕ ИССЛЕДОВАНИЯ

Наш обзор литературы и теории, посвященный гибкой разработке, демонстрирует важность продолжения исследований по этой теме. Хотя наш теоретический обзор литературы подтверждает ряд рекомендаций относительно гибкой разработки в крупных организациях, необходимы дополнительные эмпирические исследования для их проверки и дальнейшего изучения причин и следствий успешного применения гибких методов разработки в зрелых организациях.

Наша теоретическая модель и рекомендации также основаны на ограниченном наборе фундаментальных факторов, влияющих на выбор методологии. Мы признаем, что на успех ИТ-проекта также влияют многие другие факторы, описанные в существующей литературе, такие как командная культура [Walsham, 2002], поддержка со стороны высшего руководства [Schmidt et al., 2001] и соответствие организационной стратегии [Slaughter et al., 2006]. Дальнейшие исследования должны показать, как эти факторы также влияют на выбор методологии разработки ПО.

В будущих исследованиях необходимо также изучить успешность гибкой разработки как функции плотности сети консультантов проектной команды, с маягаемой затратами на поддержание неформальных отношений. Существующие литературные данные и теоретические исследования показывают, что поддержка организации и неформального и формального общения влияет на результаты используемых типов жизненных циклов разработки. Аналогичным образом, исследователям следует изучить, может ли сопоставление ролей с взаимными зависимостями маягачить влияние слабой социальной сети, и могут ли инвесторы в системы управления знаниями для проектных проектов маягачить последствие высокой волатильности команды.

Наконец, дополнительные исследования должны эмпирически проверить предположения успешной гибкой разработки. Большинство исследований с оредоточено на ограниченном числе кейсов.

VIII. ЗАКЛЮЧЕНИЕ

Как плановые, так и гибкие методологии могут быть эффективными способами разработки программного обеспечения. Каждый метод имеет свои сильные и слабые стороны. Объединяя их для создания гибридной методологии, команды разработчиков могут устранить недостатки и создать методологию разработки, даже более эффективную, чем каждая из них по отдельности. Анализ взаимосвязей и волатильности проектов позволяет руководителю определить оптимальный тип методологии для конкретной ситуации. Соответственно, мы рекомендуем крупным, зрелым организациям использовать нашу фреймворк для выбора подходящей методологии разработки. Те, кто сталкивается с высокой неопределенностью взаимосвязей маягачить в своем программном обеспечении,

Проекты должны внедрять гибридную методологию с очеташую сильные стороны текущего жизненного цикла разработки программного обеспечения с дополнительными гибкими практиками. Гибридные методологии позволяют крупным, зрелым организациям использовать преимущества гибкой разработки в областях, где ранее гибкие методологии в чистом виде не приносили успеха.

ССЫЛКИ

Примечание редактора: Следующий список литературы содержит гиперссылки на страницы Всемирной паутины. Читатели, имеющие доступ к Интернету непосредственно из текстового редактора или читающие статьи в Интернете, могут получить прямой доступ к этим ссылкам. Однако читатели должны быть предупреждены о следующем:

1. Эти ссылки существовали на дату публикации, но их работоспособность после этого не гарантируется.
2. Содержание веб-страниц может меняться со временем. В тех случаях, когда в разделе «Ссылки» указана информация о версиях, другие версии могут не содержать указанную информацию или выводы.
3. Автор(ы) веб-страниц, а не AIS, несет(ют) ответственность за точность их содержания.
4. Автор(ы) данной статьи, а не AIS, несет(ют) ответственность за точность URL и версий информации.

Абрахамсон, П., К. Конбой и Х. Ван (2009) «Много сделано, еще больше предстоит сделать: текущее состояние исследований в области разработки гибких систем», Европейский журнал информационных систем (18)4, с. 281–284.

Эмблер, С.В. (2008) «Масштабирование Scrum: удовлетворение реальных потребностей в разработке», Журнал доктора Добра (33)5, с. 52–54.

Остин, Р. и Л. Девин (2009) «Взвешивание преимуществ и издержек гибкости при создании программного обеспечения: к теории непредвиденных обстоятельств, определяющих процессы разработки», Information Systems Research (20)3, с. 462–477.

Бардхан, И.Р. и др. (2010) «Междисциплинарный подход к управлению ИТ-услугами и науке о сервисах», Журнал систем управления информационной (26)4, с. 13–64.

Бек, К. (1999) «Принятие изменений с помощью экстремального программирования», IEEE Computer (32)10, с. 70–77.

Бек, К. и К. Андрес (2004) Объяснение экстремального программирования: примите изменения, 2-е издание, Бостон, Массачусетс: Addison-Wesley Professional.



- Бек, К. и др. (2001) «Манифест Agile: Манифест гибкой разработки программного обеспечения», Agile Alliance, <http://agilemanifesto.org> (по состоянию на 8 февраля 2011 г.).
- Бергер, Х. и П. Бейнон-Дэвис (2009) «Польза быстрой разработки приложений в крупномасштабных сложных проектах», Журнал информационных систем (19)6, с. 549-570.
- Бём, Б. и Р. Тернер (2003) «Использование риска для баланса гибких и плановых методов», IEEE Computer (36)6, с. 57-66.
- Бём, Б. и Р. Тернер (2005) «Проблемы управления при внедрении гибких процессов в традиционных организациях по развитию», IEEE Software (22)5, с. 30-39.
- Бём, Б.В. (2010) «Таблица решений, учитывающая риски, для выбора процесса разработки программного обеспечения», Труды Международной конференции 2010 года по новым концепциям моделирования для современных процессов разработки программного обеспечения: процесс разработки программного обеспечения, Падерборн, Германия, с. 1.
- Бауэрс, Дж. и др. (2002) «Адаптация XP для разработки критически важного программного обеспечения для крупных задач», Труды Второй конференции XP Universe и первая конференция Agile Universe, Чикаго, Иллинойс, с. 100-111.
- Брейтуэйт, К. и Т. Джойс. (2005) «XP Expanded: Распределённое экстремальное программирование», Труды 6-й Международной конференции по экстремальному программированию и гибким процессам в программной инженерии, Шеффилд, Великобритания, с. 180-188.
- Као, Л., К. Мохан и Б. Рамеш (2004) «Насколько экстремальным должно быть экстремальное программирование? Адаптация методов XP к крупномасштабным проектам», Труды 37-й Гавайской международной конференции по системным наукам, Гавайи, с. 1-10.
- Цао, Л. и др. (2009) «Структура адаптации гибких методологий разработки», Европейский журнал Информационные системы (18)4, с. 332-343.
- Кокберн, А. и Дж. Хэйсмит (2001) «Гибкая разработка программного обеспечения: человеческий фактор», IEEE Computer (34)11, с. 131-133.
- Кон, М. (2007) «Советы по проведению Scrum of Scrums», Scrum Alliance, <http://www.scrumalliance.org/articles/46-advice-on-conducting-the-scrum-of-scrums-meeting> (по состоянию на 8 февраля 2011 г.).
- Конбой, К. (2009) «Гибкость с первых принципов: реконструкция концепции гибкости в информационных системах» «Развитие», Исследования информационных систем (20)3, с. 329-354.
- Деннис, А.Р., Р.М. Фуллер и Дж.С. Валачик (2008) «Средства массовой информации, задачи и коммуникационные процессы: теория «Синхронность СИ», MIS Quarterly (32)3, с. 575-600.
- Дональдсон, Л. (2001) Теория непредвиденных обстоятельств в организации, Лондон, Англия: Sage Publications.
- Дыба, Т. и Т. Дингс-ойр (2008) «Эмпирические исследования гибкой разработки программного обеспечения: систематический обзор», Информационные и программные технологии (50)9-10, с. 833-859.
- Эрикссон, Дж., К. Лийтinen и К. Сиау (2005) «Гибкое моделирование, гибкая разработка программного обеспечения и экстремальное программирование: состояние исследований», Журнал управления базами данных (16)4, с. 88-100.
- Фарадж, С. и С. Ли (2000) «Координация экспертных знаний в группах разработчиков программного обеспечения», журнал «Управленческие науки» (46)12, с. 1554-1568.
- Фицджеральд, Б., Г. Хартнетт и К. Конбой (2006) «Адаптация гибких методов к практикам разработки программного обеспечения в Intel» «Шеннон», Европейский журнал информационных систем (15)2, с. 200-213.
- Гэлбрейт, Дж. Р. (1973) Проектирование сложных организаций, Реддинг, Массачусетс: Addison-Wesley.
- Харрис, М., Р. Коллинз и А. Хевнер (2009) «Управление разработкой гибкого программного обеспечения в условиях неопределённости», Исследования информационных систем (20)3, с. 400-419.
- Холмстром, Х. и др. (2006) «Гибкие методы с окращивающими дистанциями глобальной разработке программного обеспечения», Информационное Управление системами (23)3, с. 7-18.
- Какконен, Т. (2004) «Гибкие методы для крупных организаций: создание сообществ практиков», Труды 2-й ежегодной конференции по гибкой разработке, Солт-Лейк-Сити, Юта, с. 2-10.
- Карлссон, Ф. и П. Агерафальк (2009) «Изучение ценностей Agile в конфигурации методов», Европейский журнал Информационные системы (18)4, с. 300-316.
- Кит, М., Х. Демirkan и М. Гул. (2009) «Разработка сервис-ориентированного программного обеспечения», Труды 15-й конференции Американская конференция по информационным системам, Сан-Франциско, Калифорния, с. 1-10.

- Кит, М. Х. Демиркан и М. Г. Оул (2010) «Влияние знаний о совместных технологиях на структуры консультационных сетей», Системы поддержки принятия решений (50)1, с. 140–151.
- Кеттунен, П. (2009) «Использование ключевых уроков гибкого производства для гибкой разработки программного обеспечения — Сравнительное исследование», Technovation (29)6–7, с. 408–422.
- Кляйн, К. Дж. и др. (2004) «Как они этого достигают? Исследование предположений центральной роли в команде» «Сети», Журнал Академии управления (47)6, с. 952–963.
- Краут, Р. Э. и Л. А. Стритер (1995) «Координация в разработке программного обеспечения», Сообщения ACM (38)3, с. 69–81.
- Куссмауль, К., Р. Джексон и Б. Спэнс-Лер (2004) «Аутсорсинг и офшоринг с Agility: пример из практики» в книге Занннер, К., Х. Эрдогмус и Л. Линдстром (ред.) Экстремальное программирование и гибкие методы — XP/Agile Universe 2004, Гейдельберг, Германия: Springer, с. 147–154.
- Ли, Г. и В. Ся (2010) «На пути к Agile: комплексный анализ количественных и качественных полевых данных о гибкости разработки программного обеспечения», MIS Quarterly (34)1, с. 87–114.
- Линдвалл, М. и др. (2004) «Гибкая разработка программного обеспечения в крупных организациях», IEEE Computer (37)12, с. 26–34.
- Мангаладж, Г., Р. Махатапра и С. Нерур (2009) «Принятие инноваций в процессах разработки программного обеспечения — случай экстремального программирования», Европейский журнал информационных систем (18)4, с. 344–354.
- Марупинг, Л., В. Венкатеш и Р. Агвал (2009а) «Теория управления: взгляд на использование гибкой методологии и «Изменение требований пользователей», Исследования информационных систем (20)3, с. 377–399.
- Марупинг, Л., С. Чжан и В. Венкатеш (2009b) «Роль коллективной ответственности и стандартов кодирования в координации экспертных знаний в командах разработчиков программного обеспечения», Европейский журнал информационных систем (18)4, с. 355–371.
- МакЭвой, Дж. и Т. Батлер (2009) «Роль управления проектами в неэффективном принятии решений в рамках проектов гибкой разработки программного обеспечения», Европейский журнал информационных систем (18)4, с. 372–383.
- Нонака, И. (1994) «Динамическая теория создания организационных знаний», Организационная наука (5)1, с. 14–37.
- Панзар, Дж. К. и Р. Д. Уиллиг (1981) «Экономия за счет масштаба», American Economic Review (71)2, с. 268–272.
- Пиккарайнен, М. и др. (2008) «Влияние гибких методов на коммуникацию в разработке программного обеспечения», Эмпирический Программная инженерия (13)3, с. 303–337.
- Пиккарайнен, М. и У. Пассоя (2005) «Подход к оценке прикладных гибких решений: пример из практики», Труды 6-й Международной конференции по экстремальному программированию и гибким процессам в программной инженерии, Шеффилдский университет, Великобритания, с. 171–171.
- Порт, Д. и Т. Буй (2009) «Моделирование смешанных гибких и плановых стратегий приоритизации требований: доказательство «Концепция и практические последствия», Европейский журнал информационных систем (18)4, с. 317–331.
- Райзинг, Л. и Н. С. Янофф (2000) «Процессы разработки программного обеспечения Scrum для небольших команд», IEEE Software (17)4, с. 26–32.
- Ройс, У. В. (1970) «Управление разработкой крупных программных систем: концепция и методы», Труды 9-й Международной конференции по программной инженерии, Монтерей, Калифорния, с. 328–338.
- Саркер, С. (2009) «Изучение гибкости в командах разработчиков распределенных информационных систем: интерпретативное исследование в «Контексте офшоринга», Исследования информационных систем (20)3, с. 440–461.
- Саркер, С., К. Мансон и С. Чакраборти (2009) «Оценки относительного вклада аспектов гибкости в успешность разработки распределенных систем: подход на основе аналитической иерархии», Европейский журнал информационных систем (18)4, с. 285–299.
- Шмидт, Р. и др. (2001) «Определение рисков программного проекта: международное исследование Дельфи», Журнал управления Информационные системы (17)4, с. 5–36.
- Швабер, К. и М. Бидл (2002) Гибкая разработка программного обеспечения с использованием Scrum, Аннер Сэддл Ривер, НьюДжерси: Prentice-Hall.
- Селик, Б. (2009) «Agile Documentation, кто-нибудь?» IEEE Software (26)6, с. 11–12.
- Слотер, С. А. и др. (2006) «Согласование процессов разработки программного обеспечения с стратегией», MIS Quarterly (30)4, с. 891–918.



Саверленд, Дж. и др. (2007) «Распределённый Scrum: гибкое управление проектами с аутсорсинговыми командами разработки», Труды 40-й ежегодной Гавайской международной конференции по системным наукам, Гавайи, с. 271–283.

Томпсон, Дж. Д. (1967) Организация и действия, Нью-Йорк: McGraw-Hill.

Турк, Д., Р. Франс и Б. Румпе (2005) «Предположения, лежащие в основе гибких процессов разработки программного обеспечения», Журнал управления базами данных (16)4, с. 62–87.

Ташман, М.Л. и Д.А. Надлер (1978) «Обработка информации как интегрирующая концепция в организационной «Дизайн», Обзор Академии управления (3)3, с. 613–624.

Виджен, Р. и Х. Ван (2009) «Совместно развивающиеся системы и организация гибкой разработки программного обеспечения», Исследования информационных систем (20)3, с. 355–376.

Уолшем, Г. (2002) «Кросс-культурное производство и использование программного обеспечения: структурный анализ», MIS Quarterly (26)4, с. 359–380.

ПРИЛОЖЕНИЕ А

Таблица А-1: Предыдущие исследования гибких методов

	Экспериментальное исследование	Построение теории	Наука о дизайне	Обзор/Комментарий
Agile Ни один из них не известен	Бергер и Бейнон-Дэвис, 2009 Бауэрс и др., 2002 Брейтуэйт и Джойс, 2005 Цао и др., 2004 Цао и др., 2009 Конбой, 2009 Фицджеральд и др., 2006 Какконен, 2004 Куссмауль и др., 2004 Ли и Ся, 2010 Линдвалл и др., 2004 Мангаларадж и др., 2009 Марупинг и др., 2009а Марупинг и др., 2009б Пиккарарайнен и Пасся, 2005 Пиккарарайнен и др., 2008 Райзинг и Янофф, 2000 Саркер, 2009 Селич, 2009 Саверленд и др., 2007 Турк и др., 2005	Остин и Девин, 2009 Харрис и др., 2009 Саркер, 2009 Виджен и Ван, 2009	Эмблер, 2008 Бек, 1999 Бауэрс и др., 2002 Брейтуэйт и Джойс, 2005 Цао и др., 2009 Порти Буй, 2009 Райзинг и Янофф, 2000 Швабери Бидл, 2002	Абрахамсон и др., 2009 Кокберн Хайсмит, 2001 Дыбаи Дингс и др., 2008 Эрикссон и др., 2005 Холмстром и др., 2006 Кеттунен, 2009
Гибридный	Порти и Буй, 2009	Цао и др., 2009	Кейт и др., 2009	Бейн Тернер, 2005

ОБ АВТОРАХ

Джордан Б. Барлоу — выпускник магистерской программы по информационным системам Школы управления Марриотта.

Руководство Университетом имени Бригама Янга, где он обучался по программе подготовки докторантов по информационным системам. Осенью 2011 года он приступит к обучению докторантуре по информационным системам в Университете Индианы. Его исследовательские интересы включают виртуальную коммуникацию с овмestную работу, человеко-компьютерное взаимодействие (HCI), управление знаниями, доверие, а также ИТ-безопасность и риски. Он работал в сфере ИТ-обучения и управления в Миссионерском учебном центре СД и был научным сотрудником в Университете Бригама Янга (BYU). В настоящее время он работает консультантом по малому бизнесу и технологиям в компании Squire & Co., PC, в Ореме, штат Юта.

Джастин Скотт Гибони — выпускник программы магистратуры по информационным системам Школы менеджмента Марриотта Университета имени Бригама Янга, где он также прошел программу подготовки к докторской диссертации по информационным системам. В настоящее время он работает над докторской диссертацией по информационным системам управления в Колледже менеджмента Эллера Университета Аризоны. Его исследовательские интересы включают доверие, ожидания при взаимодействии с компьютерами и групповое сотрудничество.

Марк Джеффри Кит — доцент кафедры компьютерной информатики и управления решениями в бизнес-колледже Западно-Техасского университета A&M. Он получил докторскую степень по информационным системам в Университете штата Аризона. Он получил степень бакалавра наук по информационным системам и степень магистра по управлению информационными системами (во время обучения в аспирантуре) в Университете имени Бригама Янга. Его последние научные интересы касаются эффективности ИТ-проектов, включая влияние консультативных сетей и сервисно-ориентированных принципов успешности проектов, затрат на переключение ИТ-решений и мобильных вычислений. Его исследования публиковались в журналах «Journal of the Association for Information Systems», «Decision Support Systems», «Decision Analysis» и «International Journal of Human-Computer Studies».

Дэвид У. Уилсон — выпускник магистерской программы по информационным системам Школы менеджмента Марриотта Университета имени Бригама Янга, где он также прошел программу подготовки к докторской диссертации по информационным системам. Он также является докторантом кафедры предпринимательства и информационных систем Университета штата Вашингтон. Его исследовательские интересы включают социальные компьютерные системы, овмestную работу, доверие и человеко-компьютерные взаимодействия.

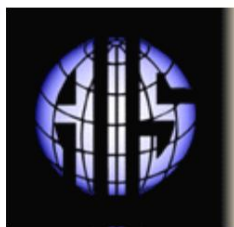
Райан М. Шуцлер — выпускник магистратуры по информационным системам Школы менеджмента Марриотта Университета имени Бригама Янга, где он также прошел программу подготовки к докторской диссертации по информационным системам. В настоящее время он работает над докторской диссертацией по информационным системам управления в Колледже менеджмента Эллера Университета Аризоны. Его исследовательские интересы включают доверие и недоверие, компьютерное взаимодействие и обнаружение обмана.

Пол Бенджамин Лоури — доцент кафедры информационных систем Гродского университета Гонконга. Его научные интересы включают взаимодействие человека и компьютера (HCI) (сотрудничество, культура, коммуникация, внедрение, развлечения, поддержка принятия решений, обман), электронную коммерцию (конфиденциальность, безопасность, доверие, брендинг, электронные рынки) и науку метрики и следований информационных систем. Он получил докторскую степень в области систем управления информацией в Университете Аризоны. Его статьи публиковались в журналах MIS Quarterly, Journal of Management Information Systems, Journal of the Association for Information Systems, Information Systems Journal, European Journal of Information Systems, Communications of the ACM, Information Sciences, Decision Support Systems, IEEE Transactions on Systems, Man, and Cybernetics, IEEE Transactions on Professional Communication, Small Group Research, Expert Systems with Applications, Communications of the Association for Information Systems и других. В настоящее время он является приглашенным помощником редактора журнала MIS Quarterly, а также регулярно назначается помощником редактора в журналах European Journal of IS, Electronic Commerce Research and Applications, Communications of the Association for Information Systems и AIS Transactions on HCI.

Энтони Вэнс — доцент кафедры информационных систем в Школе менеджмента Марриотта Университета имени Бригама Янга. Он получил степень доктора философии (PhD) по информационным системам в Университете штата Джорджия (США), Университете Париж-Дофин (Франция) и Университете Оулу (Финляндия). Он получил степень бакалавра наук (BS) в области информационных систем (IS) и магистратуры (Master of Information Systems Management, MISM) в Университете имени Бригама Янга, во время обучения в котором он был зачислен на программу подготовки докторантов (PhD) по информационным системам (IS). Ранее он работал приглашенным профессором-исследователем в Исследовательском центре безопасности информационных систем Университета Оулу, где он и по сей день является научным сотрудником. Он также работал консультантом по информационной безопасности в компании Deloitte. Его работы публикуются в журналах MIS Quarterly, Journal of Management Information Systems и European Journal of Information Systems. Его исследовательские интересы включают информационную безопасность, доверие к информационным системам и внутренний контроль.



Авторские права © 2011 принадлежат Ассоциации информационных систем. Разрешение на создание цифровых или печатных копий всей или части данной работы для личного или учебного использования предоставляется бесплатно при условии, что копии не будут создаваться и распространяться с целью получения прибыли или коммерческой выгоды, и что копии будут содержать настоящее уведомление и полную ссылку на источник на первой странице. Авторские права на компоненты данной работы, принадлежащие другим лицам, помимо Ассоциации информационных систем, должны соблюдаться. Реферирование с указанием источника разрешено. Для копирования, перепечатки, размещения на серверах или распространения по электронной почте требуется предварительное разрешение и/или плата. Запросить разрешение на публикацию можно по адресу: Административный офис AIS, почтовый индекс. Ящик 2712, Атланта, Джорджия, 30301-2712, Attn: Reprints; или по электронной почте ais@aisnet.org.



Communications of the Association for Information Systems

ISSN: 1529-3181

ГЛАВНЫЙ РЕДАКТОР
Илзе Зигурс
Университет Небраски в Омахе

КОМИТЕТ ПО ПУБЛИКАЦИЯМ AIS

Калле Лютинен Вице-президент по публикациям Университет Кейс-Вестерн Резерв	Илзе Зигурс Редактор CAIS Университет Небраски в Омахе	Ширли Грегор Редактор AIS Австралийский национальный университет
Роберт Змуд Председатель AIS региона 1 Университет Оклахомы	Филипп Эйн-Дор Председатель AIS региона 2 Тель-Авивский университет	Бернард Тан Председатель AIS региона 3 Национальный университет Сингапура

КОНСУЛЬТАТИВНЫЙ СОВЕТ CAIS

Гордон Дэвис Университет Миннесоты	Кен Крамер Калифорнийский университет в Ирвайне	Университет М. Линн Маркус Бентли,	Южный методистский университет имени Ричарда Мейсона
Джей Нунамейкер Университет Аризоны	Хенк Сол Университет Гронингена	Гавайский университет имени Ральфа С. Берджесса	Университет Техасского университета имени Хьюджа Уотсона

СТАРШИЕ РЕДАКТОРЫ CAIS

Стив Альтер Университет Сан-Франциско	Джейн Федорович Университет Бентли	Джерри Лифтман Технологический институт Стивенса
--	---------------------------------------	---

РЕДАКЦИОННЫЙ СОВЕТ CAIS

Моника Адыя Университет Маркетта	Мишель Авиталь Амстердамский университет	Динеш Батра Флорида Интернешнл Университет	Индранил Бозе Университет Гонконга
Томас Кейс Южная Джорджия Университет	Университет Вест-Индии Эвана Дагана	Мэри Грейнджер Джордж Вашингтон Университет	Оке Гронлунд Университет Умео
Дуглас Гавелка Университет Майами	Университет штата Вашингтон К. Д. Джоши,	Мишель Калика Парижский университет Дофина	Карл Айнц Каутц Копенгаген Бизнес Школа
Джули Кендалл Университет Ратгерса	Университет штата Вашингтон Нэнси Лэнкстон, Университет	Клаудия Лёббекке Маршалловский университет	Пол Бенджамин Лоури Городской университет Хонга Конг
Сал Марч Университет Вандербильта	Дон Маккаббри Денверский университет	Университет Фреда Нидермана в Сент-Луисе	Шан Лин Пан Национальный университет Сингапура
Катя Пассерини Институт НьюДжерси Технология	Ян Рекер Университет Квинсленда Технология	Джеки Риз Университет Пердью	Радж Шарман Государственный университет Нью Йорка в Буффало
Микко Сипонен Университет Оулу	Томпсон Тео Национальный университет Сингапура	Университет Святого Фомы Челли Викьяна	Падмал Витхара Сиракузский университет
Рольф Виганд Университет Арканзаса, Литл-Рок	Фонс Вейнхузен Университет Твенте	Вэнс Уилсон Вустерский политехнический институт Институт	Яцзюн Сю Университет Восточной Каролины

ОТДЕЛЫ

Информационные системы и здравоохранение Редактор: Вэнс Уилсон	Информационные технологии и системы Редакторы: Сал Марч и Динеш Батра	Статьи на французском языке Редактор: Мишель Калика
---	--	--

АДМИНИСТРАТИВНЫЙ ПЕРСОНАЛ Джеймс П. Тинсли

Исполнительный директор AIS	Випин Арора Главный редактор CAIS Университет Небраски в Омахе	Шэри Хронек Редактор публикаций CAIS Hronek Associates, Inc.	Авторское редактирование: Издательство S4Carlisle Услуги
--------------------------------	--	--	--

